

ADR-4 Estrategia de autenticación y autorización

Situación

Aceptado.

Contexto

Geojana requiere definir cómo se gestionará la autenticación de usuarios y la autorización por roles dentro del sistema. La plataforma contempla tres roles definitivos: **Usuario base**, **Institución** y **Administrador**, cada uno con permisos distintos sobre reportes, mapas, dashboard, usuarios, instituciones, contenido educativo y puntos de interés.

Esta decisión se vuelve necesaria porque el sistema debe restringir funcionalidades y datos según el perfil del usuario. En el planteamiento del proyecto se define que Geojana debe implementar roles para que cada perfil solo ejecute operaciones autorizadas, permitir el registro de usuarios base, y restringir la creación de perfiles administrativos e institucionales al módulo de administración .

Además, la arquitectura ya definida establece que Geojana utilizará una **Arquitectura Basada en Servicios**, donde las PWAs no consumen directamente los servicios internos, sino que se comunican mediante una **Capa de API o Gateway** . En ADR-6 también se define un **Identity & Access Service** encargado de gestionar identidad, perfiles, roles, permisos, puntos de confianza y estados de cuenta .

El sistema también debe ser compatible con funcionamiento offline parcial, especialmente para el registro de reportes en zonas rurales. Sin embargo, las operaciones sensibles, como gestionar instituciones, modificar roles, actualizar contenido, cambiar estados de reportes o acceder a dashboards institucionales, deben requerir validación de identidad y permisos en línea.

Decisión

Se decide utilizar **Supabase Auth + JWT**, complementado con un **Identity & Access Service** propio para manejar autorización por roles y permisos dentro de Geojana.

La autenticación será delegada a **Supabase Auth**, mientras que la autorización específica del dominio será gestionada por el backend de Geojana mediante el **Identity & Access Service**. Esta decisión se alinea con ADR-3, donde Supabase ya fue seleccionado como parte del stack tecnológico por su integración con PostGIS, autenticación y políticas RLS . También se alinea con ADR-8, donde se propone usar Supabase Storage y Auth para identidad centralizada, validación de JWT desde el API Gateway y protección de evidencia multimedia mediante RLS .

La responsabilidad quedará distribuida de la siguiente manera:

Componente	Responsabilidad
Supabase Auth	Registro, inicio de sesión, emisión de access tokens y refresh tokens.
JWT	Transportar la identidad del usuario en las solicitudes hacia el API Gateway.
API Gateway	Validar el token, aplicar reglas de entrada, rechazar solicitudes no autorizadas y

	enrutar al servicio correspondiente.
Identity & Access Service	Gestionar perfiles, roles, permisos, puntos de confianza, estados de cuenta y asociación de usuarios institucionales.
Servicios de dominio	Ejecutar reglas propias del negocio, confiando en la identidad validada por el Gateway.
Supabase RLS	Proteger datos sensibles, especialmente evidencia fotográfica y datos asociados a instituciones.

El flujo general será:

1. El usuario inicia sesión desde la PWA ciudadana o la plataforma institucional/administrativa.
2. Supabase Auth valida las credenciales.
3. Supabase entrega un JWT al frontend.
4. El frontend envía sus solicitudes al API Gateway incluyendo el token.
5. El API Gateway valida firma, expiración y datos mínimos del JWT.
6. El Gateway consulta o utiliza información del Identity & Access Service para verificar permisos.
7. Si el usuario tiene autorización, la solicitud se redirige al servicio correspondiente.
8. Si no tiene autorización, el sistema responde con error 401 o 403, según corresponda.

Los roles definitivos serán:

Rol	Permisos principales
Usuario base	Registrarse, iniciar sesión, crear reportes, consultar mapa, consultar guías, validar reportes cercanos, marcar reportes falsos o duplicados, consultar historial y puntos de confianza.
Institución	Consultar reportes derivados, ver detalle de reportes, cambiar estados, agregar comentarios de gestión, visualizar dashboard institucional y exportar datos autorizados.
Administrador	Gestionar usuarios, instituciones, roles, puntos de interés, contenido educativo, guías, fauna, reportes conflictivos y cuentas suspendidas o bloqueadas.

Se recomienda que los roles sean valores controlados, por ejemplo:

```
base_user
institution
admin
```

Además, se recomienda complementar los roles con permisos específicos, tales como:

```
report:create
report:validate
report:update_status
report:moderate
institution:manage
poi:manage
content:manage
dashboard:view
users:manage
```

Esto evita que la autorización dependa solo de tres roles rígidos y permite extender permisos en el futuro sin rediseñar toda la estrategia de seguridad.

Consecuencias

La decisión presenta los siguientes trade-offs:

Aspecto	Consecuencia
Seguridad	Positiva. Supabase Auth reduce el riesgo de implementar autenticación manualmente. El uso de JWT, expiración y refresh token entrega una base segura para APIs.
Costo operativo	Positiva. No se requiere desplegar ni mantener un servidor de identidad complejo como Keycloak.
Complejidad de implementación	Positiva. Es más simple que una autenticación propia o Keycloak, y se integra con el stack ya seleccionado.
Compatibilidad con API Gateway	Positiva. El Gateway puede validar JWT antes de redirigir solicitudes a los servicios internos.
Compatibilidad offline parcial	Positiva con límites. La PWA puede conservar una sesión previamente iniciada y encolar reportes offline, pero no debe permitir operaciones administrativas sin conexión.
Roles y permisos granulares	Positiva. Los roles principales se gestionan en Identity & Access Service y los permisos específicos pueden crecer gradualmente.
Dependencia de proveedor	Negativa. Se genera dependencia parcial de Supabase para autenticación, sesiones y

	políticas RLS.
Tokens expirados offline	Negativa. Si el token expira mientras el usuario está sin conexión, la app deberá conservar los datos localmente y pedir reautenticación antes de sincronizar.
Control absoluto de identidad	Negativa frente a una auth propia. Geojana delega parte del control a Supabase, aunque reduce carga técnica y riesgos de seguridad.

Alternativas evaluadas

Alternativa	Evaluación
Supabase Auth + JWT	Alternativa seleccionada. Tiene baja complejidad, bajo costo, integración directa con Supabase/PostgreSQL/RLS, buen soporte para API Gateway y seguridad suficiente para el alcance del proyecto.
Auth propia con FastAPI/Django	Ofrece control total, pero aumenta mucho la responsabilidad de seguridad: hashing, refresh tokens, recuperación de contraseña, expiración, revocación y protección ante ataques. No es conveniente para un equipo con tiempo limitado.
Keycloak	Muy robusto y adecuado para entornos empresariales, SSO y OAuth2/OIDC avanzado. Sin embargo, es demasiado complejo para Geojana y aumenta la carga operativa.
Firebase Authentication	Fácil de usar y compatible con aplicaciones web/móviles, pero menos coherente con la decisión previa de usar Supabase, PostgreSQL y PostGIS. Implicaría mezclar proveedores sin necesidad.
Sesiones tradicionales	Modelo conocido, pero menos adecuado para una arquitectura basada en servicios, API Gateway y PWA offline parcial. Requiere mayor manejo de cookies, CSRF y estado de sesión.

La opción seleccionada equilibra seguridad, bajo costo, simplicidad operativa y coherencia con las decisiones arquitectónicas ya aceptadas.

Cumplimiento

El cumplimiento de esta decisión se verificará mediante las siguientes métricas y pruebas:

Criterio	Forma de verificación
Tiempo promedio de autenticación	El inicio de sesión debe completarse en menos de 2 segundos en una red normal.
Validación de JWT en Gateway	Toda petición protegida sin token, con token inválido o expirado debe responder 401.
Autorización por rol	Usuario base, Institución y Administrador solo deben acceder a las funcionalidades permitidas para su rol.
Permisos granulares	Se deben probar permisos como <code>report:update_status</code> , <code>users:manage</code> , <code>poi:manage</code> y <code>content:manage</code> .
Compatibilidad offline parcial	La PWA debe permitir crear un reporte offline, guardarlo en IndexedDB y sincronizarlo al recuperar conexión.
Bloqueo de acciones sensibles offline	No se deben permitir acciones administrativas o institucionales críticas sin conexión.
Estados de cuenta	Una cuenta suspendida o bloqueada no debe poder operar aunque conserve una sesión previa.
Protección de evidencia multimedia	Las imágenes de reportes deben ser visibles solo para la institución competente y el administrador mediante políticas RLS.
Acceso centralizado	El frontend debe consumir únicamente el API Gateway y no llamar directamente a servicios internos.
Refresh y expiración de tokens	El sistema debe renovar sesión cuando corresponda y solicitar reautenticación si el token ya no es válido.

Notas

Autor: Belén Bravo Sepúlveda - b.bravo02@ufromail.cl

Día de publicación: 06-06-2026

Última actualización: 06-06-2026